Heriot-Watt University
School of Engineering & Physical Sciences
Electrical, Electronic and Computer Engineering

B38RO – Introduction to Robotics

Lab Manual

By: Walid Shaker

Email: wkhs2000@hw.ac.uk

Supervised by: Mustafa Suphi Erden

Project Overview

1. Objective

This project aims to design, simulate, and implement an **Automated Sorting Station** capable of identifying, sorting, and placing colored objects in their designated areas. Students will first develop and test their solution in a simulation environment followed by a real-world experimentation on robotic hardware.

2. Learning Outcomes

By completing this project, students will:

- Gain experience programming robotic systems in simulation and hardware.
- Apply and practice theoretical concepts studied throughout the course material.
- Learn to integrate sensors (IR and camera) for object detection and their application in robotic automation systems.
- Implement robotic manipulation for tasks such as sorting, picking, and placing.

3. Equipment and Software

- **Simulation Software:** RoboDK[1], an offline programming and simulation software.
- **Hardware:** Niryo[2] (Ned2 robot, camera, conveyor belt, IR sensor, and colord objects).
- **Programming Environment:** Python.

4. Project Work

The project is divided into two sections: simulation and hardware. It will be carried out in groups of three students. If any student is uncomfortable working in a group, they should contact the instructor to discuss alternative arrangements.

In both sections, the automated sorting station should function as follows:

1. **Object Placement**: The station involves three pairs of colored pieces: red, green, and blue. The user starts by randomly selecting a piece and placing it on the conveyor belt.

[1] https://robodk.com/
[2] https://niryo.com/

2. **Detection by IR Sensor**: An IR sensor positioned at the end of the conveyor detects the object. Once detected, the conveyor stops.

3. **Robot Observation**: The robot moves to an observation pose where the part is detected.

4. **Color and Pose Identification**: Using a camera mounted on the robot end-effector, it identifies the object's color and determines its picking pose.

5. **Picking and Sorting**: After identifying the object, the robot picks it up and places it into the corresponding colored area.

5. Reporting Requirements

At the end of the project, students are required to submit the following:

1. **Demos**:
   - Two demos for both the simulation and hardware implementations.
2. **Report**:
   - A 5-10 pages report that includes:
     1. A brief description of the project workflow, including both the simulation and hardware setups.
     2. A reflection on how the theoretical material learned in the course was applied to complete the project.
     3. Observations and results from both the simulation and hardware implementations, including relevant plots.
     4. Code snippets of the two main.py files for both simulation and hardware.
     5. A summary of each group member’s contribution to the overall project.

6. Knowledge Exchange

Students are encouraged to collaborate, share knowledge, and leverage each other's skills. However, they should avoid "copy-and-paste" practices. Instead, the focus should be on mutual learning and understanding. Each group is expected to implement, develop, and demonstrate its own work based on this knowledge exchange.

7. Assessment Criteria

The assessment of the project will consist of three components, totaling 100 points, which will contribute a percentage to the overall course grade:

1. **Demos (50%)**
2. **Report (50%)**

Simulation Guideline

1. Prerequisites

Before starting, ensure you have the following software installed on your machine:

- **RoboDK**:
  - Download the appropriate version for your operating system: RoboDK Download.
  - Install RoboDK on your machine.
- **Python**:
  - Download Python for your operating system: Python Download.
  - Install Python. For Windows users, check the option to add Python to the system environment variables.
- **Integrated Development Environment (IDE)**:
  - Download an IDE like Visual Studio Code (VS Code): VS Code Download.
  - Install VS Code and the recommended Python extensions.

2. Library Installation

Once the prerequisites are installed, follow these steps to install the necessary Python libraries:

- Open the **Command Prompt (Windows)**.
- Upgrade pip and install the required libraries:

```
python.exe -m pip install --upgrade pip
pip install robodk
pip install numpy
pip install opencv-python
pip install matplotlib
```

3. Download Required Files

Download the following files from the **course resources on Canvas**:

- **RoboDK Station**: *Niryo.rdk*
- **Python Helper File**: *helpers_sim.py*
- **Main Python Script**: *main_sim.py*

Make sure to save these files in a dedicated project folder on your computer.

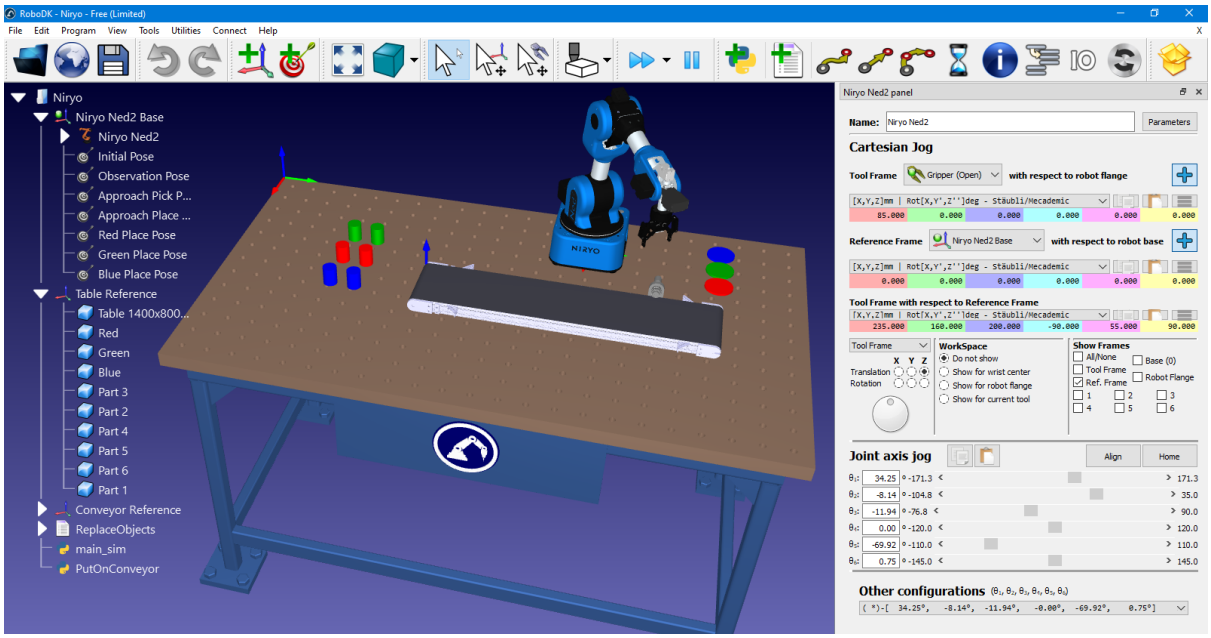
With these steps completed, you are ready to start your project simulation!

4. Getting Started with the Simulation Station

Launching the Simulation Station

Since the free version of RoboDK has limitations, follow these steps to open the provided *Niryo.rdk* file:

1. Launch the **RoboDK App**.
2. From **File > Open**, navigate to the project folder and select **Niryo.rdk**.
3. Once the file is opened, you should see the simulation station, as shown below.



On the right-hand side, the **robot panel** allows you to:

- Monitor joint angles and the tool frame relative to the robot base.
- Control the robot using **Cartesian Jog** or **Joint axis jog**.

To access the robot panel, simply double-click on the **Niryo Ned2** robot in the left-hand tree.

Station Components

The simulation station includes the following components:

- **Niryo Ned2 Robot** with a virtual camera mounted on its end-effector.
- **Conveyor Belt** equipped with an IR sensor to detect objects.
- **Colored Cylindrical Objects:** red, green, and blue.
- **Three Corresponding Placement Areas** for sorting the objects.

Predefined Saved Poses

These saved poses are available in the simulation and will be used during the project:

- **Initial Pose**: The starting position for the robot, where the program begins.
- **Observation Pose**: The position where the robot detects the object using the IR sensor.
- **Approach Pick Pose**: A transitional pose near the picking pose for smooth movement.
- **Approach Place Pose**: A transitional pose near the placing pose for smooth movement.
- **Red Place Pose**: The position where the red object should be placed.
- **Green Place Pose**: The position where the green object should be placed.
- **Blue Place Pose**: The position where the blue object should be placed.

Make sure to familiarize yourself with these poses, as they will be integral to completing the simulation tasks effectively.

5. Helper Functions

Below is a list of the functions provided in the **helpers_sim.py** file, along with their descriptions. Please note that this file should not be altered or modified.

1. **open_gripper()**

- Opens the robot's gripper.

2. **close_gripper()**

- Closes the robot's gripper on the nearest object.

3. **run_conveyor()**

- Starts the conveyor belt in the RoboDK environment.

4. **stop_conveyor()**

- Stops the conveyor belt program in RoboDK.

5. **put_on_conveyor()**

- Places an object randomly onto the conveyor belt in the simulation.

6. **detect_part()**

- Detects objects using an IR sensor by checking proximity to a predefined detection plane. It returns True if an object is detected.

7. **get_color_and_pose()**

- Captures a camera snapshot to detect the color of an object and its pose relative to the robot base frame. It returns the detected object's color and pose.

8. **get_joints()**

- Gets the current joint angles of the robot.

9. **get_pose()**

- Gets the current pose of the robot's end-effector as [x, y, z, roll, pitch, yaw].

10. **get_target_joints(target_name)**

- Retrieves the joint configuration required to move the robot to a specific target.

11. **get_target_pose(target_name)**

- Gets the Cartesian pose of a specific target as [x, y, z, roll, pitch, yaw].

12. **move_joints(q)**

- Moves the robot to a specified joint configuration along a non-linear path.

13. **move_pose(pose)**

- Moves the robot end-effector in a straight line to a specified Cartesian pose.

14. **forward_kinematics(joints)**

- Computes the end-effector pose for a given set of joint angles.

15. **inverse_kinematics(pose)**

- Computes the joint angles required for a given end-effector pose.

16. **jacobian_matrix (joints)**

- Calculates the Jacobian matrix at a given joint configuration.

17. **wait(duration)**

- Pauses the program for a specified duration (in seconds).

18. **close_connection()**

- Closes the camera and destroys any OpenCV windows used during the process.

6. Main File

The **main_sim.py** file, as shown below, is where the simulation will be implemented.

```python
from helpers_sim import *

# ------------------------------------------------------------------------------------------------------------------
# ------------------------------------------------------------------------------------------------------------------

if __name__=="__main__":

    robot = NiryoRobot()
    robot.open_gripper()

    initial_pose = robot.get_target_pose('Initial Pose')         # [380.221, 0.0, 429.030, 0.0, 0.003, 0.0]
    observation_pose = robot.get_target_pose('Observation Pose') # [235.0, 160.0, 200.0, 0.0, 90.0, 35.0]
    approach_pose = robot.get_target_pose('Approach Pose')       # [10.0, 325.0, 120.0, 0.0, 90.0, 90.0]
    red_place_pose = robot.get_target_pose('Red Place Pose')     # [80.0, 325.0, 27.5, 0.0, 90.0, 90.0]
    green_place_pose = robot.get_target_pose('Green Place Pose') # [10.0, 325.0, 27.5, 0.0, 90.0, 90.0]
    blue_place_pose = robot.get_target_pose('Blue Place Pose')   # [-60.0, 325.0, 27.5, 0.0, 90.0, 90.0]

    # --- --------- --- #
    # --- YOUR CODE --- #
    # --- --------- --- #

    robot.close_connection()

```

Students are required to complete the following tasks:

- Obtain the joint angles for the **Initial Pose** and move the robot to this configuration.
- Calculate the required joint angles to move the robot to the **Observation Pose**.
- Compute the Jacobian matrix at the **Observation Pose**.
- Place the objects onto the running conveyor.
- Detect the color and pose of each object.
- Implement a sequence where the robot:
    - Moves its end-effector in a straight line to approach the object.
    - Picks up the object.
    - Places the object in its designated **Color Place Pose** by moving the end-effector in a straight line.
    - Returns to the **Observation Pose** to repeat the process for the next object.
- After all objects are placed, move the robot to the **Initial Pose**.
- Plot the joint trajectory of the robot's motion starting from **Initial Pose** and returning to the **Initial Pose**.

Note: Students may perform intermediate steps, if needed, to ensure smooth and precise motion while avoiding potential collisions.

Hardware Guideline

1. Prerequisites

Before getting started, ensure the following software is installed on your machine:

- **NiryoStudio**:
  - Download the appropriate version for your OS: NiryoStudio Download.
  - Install NiryoStudio by following the tutorial: Start with Ned2.

2. Library Installation

- Install the PyNiryo library using the following command:
  pip install pyniryo

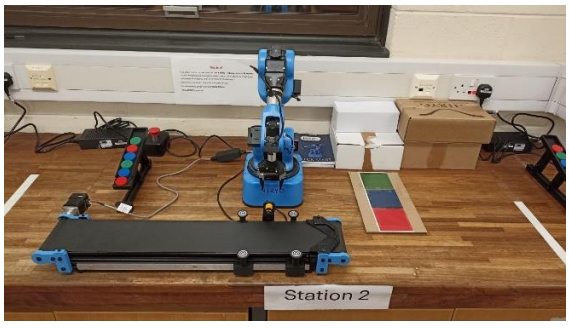
3. Download Required Files

Download the following file from the **course resources on Canvas**:

- **Main Python Script**: *main_hardware.py*

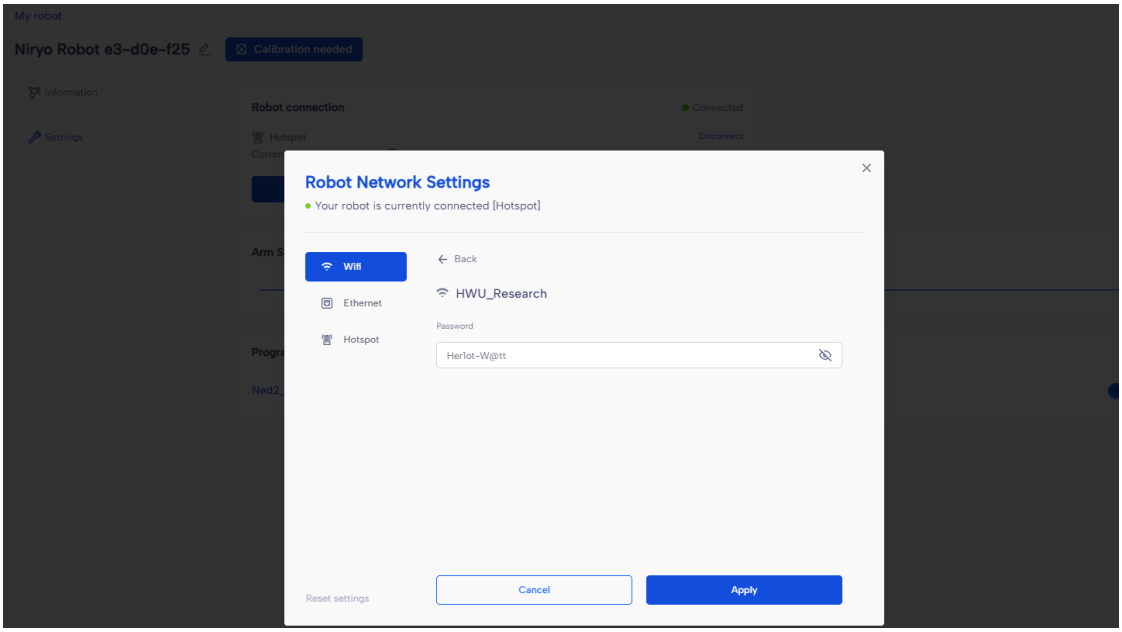
4. Getting Started with Ned2

There are four available stations, each equipped with a robot, conveyor belt, colored objects, manuals, and accessories boxes. Each group will be assigned to one station, which will be used consistently throughout the project.



To connect the robot, follow the steps outlined in the tutorial: Connect the robot.

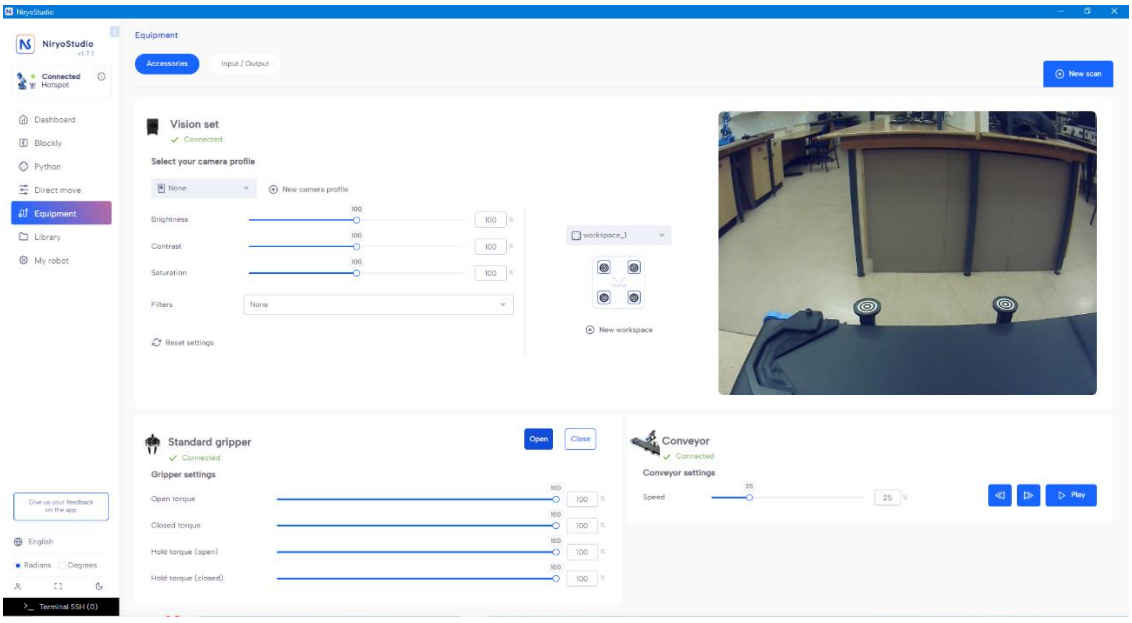
After successfully connecting in hotspot mode, you can switch to Wi-Fi mode. Note that the robots only have access to the **HWU_Research** Wi-Fi network.
Password: Her1ot-W@tt



Follow these tutorials to get familiar with robot programming using **Blockly**.

- First moves with Ned2
- Start with the grippers
- First Pick and Place
- Start with the vision set
- Start with the conveyor

Ensure the gripper and conveyor are detected, and the camera workspace is configured.



5. Main File

The **main_hardware.py** file, as shown below, is where the robot's workflow will be implemented.

main_hardware.py > ...

```python
from pyniryo import *

# ----------------------------------------------------------------------------------------------------
# ----------------------------------------------------------------------------------------------------

if __name__=="__main__":

    robot_ip_address = '10.10.10.10'
    workspace_name = "workspace_1"  # Robot's Workspace Name
    robot = NiryoRobot(robot_ip_address) # Connect to robot

    # Clear collision if detected during a previous movement
    if robot.collision_detected:
        robot.clear_collision_detected()

    robot.calibrate_auto() # Calibrate robot if the robot needs calibration
    robot.update_tool()    # Updating tool
    robot.open_gripper()

    sensor_pin_id = 'DI5'  # Setting variables
    conveyor_id = robot.set_conveyor() # Activating connexion with the Conveyor Belt

    # --- --------- --- #
    # --- YOUR CODE --- #
    # --- --------- --- #

    robot.unset_conveyor(conveyor_id) # Deactivating connexion with the Conveyor Belt
    robot.close_connection()
```

Important Notes

- Please review the PyNiryo API Documentation for a detailed understanding of available methods.
- Start by saving the necessary poses for your program.
  As a guide, you can follow this tutorial: First Automation with Blockly.
- Saved poses can be accessed using the PyNiryo method: **get_saved_pose_list()**.
- The state of the IR sensor can be read using **digital_read(pin_id)**.
- The sensor returns **PinState.HIGH** when no object is detected.
- For object detection, the following method can be helpful:
  move_to_object(*workspace_name*, *height_offset*, *shape*, *color*).
- In case of robot faults, press the robot's emergency button immediately.
- Please do not plug or unplug any cables from the robots to avoid potential damage.

Project Timeline

Week	Content
2 - 4	Simulation Part
5	Hardware Setup
6	Consolidation Week
7 – 11	Hardware Implementation
12	Review and Evaluation